Program 4 - Implementation of Support Vector Machine.

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data[:, 2:4] # Petal length and petal width
y = iris.target

# We will use only two classes (binary classification) for clear visualization
X = X[y != 2]
y = y[y != 2]

# Standardize the features (important for SVM)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Train the SVM model
model = SVC(kernel='linear')
model.fit(X, y)
```

```
# Plotting
plt.figure(figsize=(8, 6))
ax = plt.gca()
# Create a mesh to plot decision boundary
xlim = ax.get_xlim()
ylim = ax.get_ylim()
xx, yy = np.meshgrid(np.linspace(-3, 3, 500),
np.linspace(-3, 3, 500))
xy = np.c_[xx.ravel(), yy.ravel()]
Z = model.decision_function(xy).reshape(xx.shape)
# Plot decision boundary and margins
plt.contour(xx, yy, Z, colors='k', levels=[-1, 0, 1],
linestyles=['--', '-', '--'])
# Plot data points
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='coolwarm', s=30)
# Highlight support vectors
plt.scatter(model.support_vectors_[:, 0],
model.support_vectors_[:, 1],
s=100, facecolors='none', edgecolors='k')

plt.title("SVM on Iris Dataset (Petal Length vs Width)")
plt.xlabel("Petal Length (standardized)")
plt.ylabel("Petal Width (standardized)")
plt.grid(True)
plt.show()
```